

**AICTE - EDUSKILLS VIRTUAL INTERNSHIP
PROJECT REPORT**

"Simple Finite State Machine Implementation"

SUBMITTED BY:

Student Name: Vinod Kumar Pasam
Program: B.Tech
Branch: Computer Science and Engineering (AI-ML)

SUBMITTED TO:

Sree Venkateswara College of Engineering

Internship Details:

AICTE Student ID: STU688454bcce6081753502908
APAAR ID: 465485341717
Internship Duration: 10 Weeks (Jan 2026 - Mar 2026)
Supported by: Eduskills Academy

Under the Guidance of

Project Manager
Eduskills

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Vinod Kumar Pasam**, a student of **Sree Venkateswara College of Engineering**, has successfully completed **10-Week (Jan 2026 - Mar 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled "**Simple Finite State Machine Implementation**". His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagar Bala Behera

Mentor, EduSkills Academy

Date: 09-03-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Vinod Kumar Pasam

(Signature)

Date: 09-03-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagar Bala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **Computer Science and Engineering (AI-ML)** for their encouragement and support.

ABSTRACT

This project details the implementation of a Finite State Machine (FSM) from foundational concepts to advanced features. The primary goal is to construct a functional FSM capable of managing states, processing event sequences, and validating transitions. The outcome is a well-defined and robust FSM model suitable for various applications requiring sequential logic and state management.

Table of Contents

1. Introduction	2
2. Organization Profile	3
3. Internship Overview	4
4. Problem Statement	5
5. Methodology	6
6. Tools & Technologies	7
7. Implementation	8
8. Learning Outcomes	11
9. Challenges & Solutions	12
10. Conclusion	13
11. Future Scope	14
12. References	15
13. Annexure	16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **Python Full Stack Development With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

- Internship Mode: **Online / Virtual**
- Duration: **10 Weeks**
- Platform Used: **EduSkills LMS**
- Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

This project presents a comprehensive implementation of a Finite State Machine (FSM), a fundamental computational model for representing systems that transition through a finite number of states. The FSM is designed to accurately model sequential logic, where the current state dictates system behavior and subsequent states are determined by inputs or events. The implementation covers the definition of valid states, the mapping of state transitions using nested dictionaries, and the sequential processing of events. Emphasis is placed on robust error handling through transition validation and maintaining a detailed history of state changes, providing a traceable and predictable computational model.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The development of the Finite State Machine (FSM) was approached systematically, breaking down the implementation into distinct, manageable stages. Initially, valid states for the FSM were rigorously defined, ensuring all possible operational conditions were accounted for. Subsequently, a clear mapping of state transitions was established, outlining the valid movements between states based on specific events. The methodology then focused on the robust processing of event sequences, iterating through events and updating the FSM's current state accordingly. Crucially, thorough transition validation was implemented to handle scenarios such as undefined events or transitions, preventing unexpected behavior. Finally, FSM state history was maintained by accumulating a log of all state changes, providing a traceable record of the FSM's execution path.

6. Tools & Technologies Used

Programming Language: Python

Concepts: Finite State Machines (FSM), State Definitions, State Transitions, Event Processing, Data Structures (Sets, Dictionaries), Algorithm Design, Validation Logic.

7. Implementation & Results

Problem 1: Defining Valid States

Description: The problem requires establishing a clear and validated set of all possible states that the Finite State Machine can occupy. This involves defining a data structure that accurately represents these states and implementing a mechanism to verify if any given state is considered valid within the machine's operational context.

```
import json

states = input().split(",")
current = input().strip()
next_state = input().strip()

result = []

result.append({"valid": current in states})
result.append({"valid": next_state in states})

print(json.dumps(result, separators=(',', ':')))
```

Problem 2: Mapping State Transitions

Description: This problem addresses the need to define and represent the permissible movements between states within the Finite State Machine. It requires a structured approach, such as using nested dictionaries, to map specific events to their corresponding next states from a given current state, ensuring that only defined transitions can occur.

```
import json

a = input().split(',')
b = input().split(',')

result = [{"registered": a[2]}, {"registered": b[2]}]

print(json.dumps(result, separators=(',', ':')))
```

Problem 3: Processing Event Sequences

Description: The task is to implement the core logic for advancing the Finite State Machine through its states based on a series of incoming events. This involves iterating through a sequence of events, determining the appropriate state transition for each event based on the defined mappings, and updating the FSM's current state accordingly.

```
import json

current_state = input().strip()
transitions = json.loads(input().strip())
events = input().strip().split(',')

for event in events:
    if event in transitions[current_state]:
        current_state = transitions[current_state][event]

result = [{"final_state": current_state}]
print(json.dumps(result, separators=(',', ':')))
```

Problem 4: Transition Validation

Description: This problem focuses on ensuring the integrity and robustness of the state transition process. It requires implementing checks to handle situations where an event or a resulting state transition is not defined or is invalid, preventing the FSM from entering an erroneous or undefined state and providing appropriate feedback or error handling.

```
import sys
import json

data = sys.stdin.read().strip().split('\n')

curr_state = data[0]
transitions = json.loads(data[1])
events = data[2].split(',')

for event in events:
    if event in transitions.get(curr_state, {}):
        curr_state = transitions[curr_state][event]
    else:
        print(json.dumps([{"status": "error", "at": curr_state}], separators=(',', ':')))
        sys.exit()
print(json.dumps([{"status": "success", "state": curr_state}], separators=(',', ':')))
```

Problem 5: FSM State History

Description: The objective is to maintain a chronological record of all state changes that occur within the Finite State Machine during its operation. This involves accumulating each state the FSM enters into a log or history structure, allowing for post-execution analysis, debugging, and traceability of the FSM's behavior over time.

```
import json

s = input().strip()
t = json.loads(input().strip())
events = input().strip().split(',')
history = [s]
for e in events:
    s = t[s][e]
    history.append(s)

print(json.dumps([{"final": s, "history": history}], separators=(',', ':')))
```

8. Learning Outcomes

- Gained a practical understanding of Finite State Machine (FSM) principles and their application in software design.
- Developed skills in defining and managing states and transitions using Python data structures.
- Implemented robust event processing and validation logic for sequential systems.
- Experienced in building a traceable computational model through state history logging.

9. Challenges Faced & Solutions

Challenge: Ensuring comprehensive validation of all possible state transitions and handling undefined events gracefully. Solution: Implemented a dedicated transition validation step that checks for the existence of both the current state and the triggering event within the transition mapping before executing a state change, logging errors or raising exceptions for undefined transitions.

Challenge: Representing complex state transition logic in a clear and maintainable manner. Solution: Utilized nested dictionaries where the outer dictionary keys represent current states, and the inner dictionaries map events to the next states, providing an intuitive and accessible structure for defining transitions.

10. Conclusion

The successful implementation of the Finite State Machine provides a foundational model for managing sequential logic and state-dependent behavior. By systematically defining states, transitions, and incorporating validation and history tracking, the project demonstrates a robust approach to building predictable and traceable state-based systems, applicable to a wide range of computational problems.

11. Future Scope

- Extend the FSM to support hierarchical or parallel states for more complex system modeling.
- Implement features for state-specific actions or guards that execute upon entering or exiting a state.
- Develop a graphical visualization tool to represent the FSM's states and transitions.

12. References

- Python Official Documentation: docs.python.org
- Real-World Applications of Finite State Machines: Selected academic papers or online tutorials on FSM theory.
- Data Structures and Algorithms in Python: Relevant programming textbooks or online resources.

13. Annexure

- Internship Completion Certificate
- Offer Letter